



Carátula para entrega de prácticas

Facultad de Ingeniería

Laboratorios de docencia

Laboratorio de Computación Salas A y B

Profesor(a): René Adrián Dávila Pérez

Asignatura: Programación Orientada a Objetos

Grupo: Grupo 7

No de Práctica(s): Practica 1

Integrante(s): 426058744

120002850

323064039

323071378

323214636

426059576

Semestre: 3er Semestre

Fecha de entrega: 29 de agosto de 2026

Observaciones:

CALIFICACIÓN: _____

Índice

1. Introducción	1
2. Marco Teórico	1
2.1. Paradigma Orientado a Objetos e Inmutabilidad de Entidades	1
2.2. Arquitectura de Interfaces Gráficas y Programación Dirigida por Eventos	1
2.3. Modelo de Renderizado Vectorial y Rasterización en Lienzo	2
2.4. Patrón Memento y Estructuras de Datos para Reversión de Estado	2
2.5. Flujo de Control y Aritmética Computacional Defensiva	2
2.6. Ingeniería de Software Asistida por Modelos de Lenguaje	2
3. Desarrollo	2
3.1. Ejercicio 1: Arquitectura del Editor Gráfico Generado por LLM	2
3.2. Ejercicio 2: Formalización y Ejecución de la Lógica del Pizarrón	7
3.2.1. Estructura Central de Despacho	7
3.2.2. Operaciones Aritméticas	8
3.2.3. Casos excepcionales de indeterminación	8
4. Conclusión	10
Referencias	11

1. Introducción

El desarrollo de software profesional exige una comprensión profunda de dos modelos de ejecución complementarios: el flujo secuencial determinista que gobierna la lógica algorítmica y el paradigma reactivo que sustenta a las interfaces gráficas de usuario. Para un estudiante de ingeniería, comprender la transición de especificaciones abstractas a sistemas ejecutables requiere dominar tanto la gestión de eventos físicos (como las interacciones del cursor sobre un lienzo) como el diseño defensivo ante operaciones aritméticas con restricciones de dominio.

Esta práctica se justifica por la necesidad de establecer una línea base de análisis técnico en dos frentes de trabajo. En primer lugar, se examina una solución generada mediante modelos de lenguaje de gran tamaño (LLM) para un editor gráfico interactivo, analizando su arquitectura orientada a objetos y gestión de historial. En segundo lugar, se formaliza la lógica aritmética colaborativa construida en el aula, implementando una calculadora de consola que maneja rigurosamente casos excepcionales como divisiones por cero o raíces fuera del dominio real. Estas actividades sientan las bases metodológicas para contrastar posteriormente el código asistido por inteligencia artificial frente al desarrollo humano.

Para evaluar el desarrollo de estas competencias, se establecieron los siguientes objetivos:

- Analizar la arquitectura orientada a objetos de una herramienta gráfica interactiva generada por LLM, examinando su respuesta
- Implementar y verificar la lógica aritmética formulada en el pizarrón mediante una aplicación estructurada en Java, garantizando el análisis de casos excepcionales para su correcto funcionamiento.

2. Marco Teórico

2.1. Paradigma Orientado a Objetos e Inmutabilidad de Entidades

La Programación Orientada a Objetos (POO) estructura el software mediante abstracciones que encapsulan estado y comportamiento [2]. Un principio fundamental en la arquitectura de software moderna es la inmutabilidad: un objeto inmutable mantiene sus atributos constantes desde su instanciación, eliminando efectos secundarios y garantizando consistencia en entornos concurrentes. En plataformas como Java, esta propiedad se formaliza mediante tipos de datos compactos o clases finales cuyos campos no admiten reasignación [3].

2.2. Arquitectura de Interfaces Gráficas y Programación Dirigida por Eventos

A diferencia de los programas secuenciales, una interfaz gráfica opera bajo un modelo reactivo impulsado por un hilo de despacho de eventos. Según Deitel y Deitel [2], los periféricos generan interrupciones que el sistema traduce en eventos. Mediante el patrón de diseño *Observer* [1], los componentes del lienzo escuchan transiciones de estado del ratón, recalculando coordenadas sin bloquear el hilo principal.

2.3. Modelo de Renderizado Vectorial y Rasterización en Lienzo

El renderizado comprende el cálculo vectorial de primitivas geométricas y su rasterización. El contexto gráfico proporciona un búfer de dibujo donde las geometrías se proyectan mediante transformaciones afines [3]. Asimismo, la persistencia gráfica requiere exportar el búfer de memoria a formatos matriciales estándar como PNG para su visualización externa.

2.4. Patrón Memento y Estructuras de Datos para Reversión de Estado

El control de cambios (*undo*) exige almacenar el historial de mutaciones. El patrón *Memento* resuelve este requerimiento capturando instantáneas del estado interno sin violar el encapsulamiento [1]. Desde la perspectiva de estructuras de datos, este historial se gestiona mediante una colección de acceso LIFO (*Last-In, First-Out*), garantizando operaciones de apilado y desapilado en tiempo constante $O(1)$ [4].

2.5. Flujo de Control y Aritmética Computacional Defensiva

El diseño de algoritmos aritméticos se sustenta en estructuras de control selectivas e iterativas [4]. La programación defensiva postula que toda precondition matemática debe validarse previo a la ejecución:

- **División y Módulo:** El teorema del residuo establece que para $a, b \in \mathbb{Z}$ con $b \neq 0$, existen únicos enteros q y r tales que $a = b \cdot q + r$ [4].
- **Radicación:** La raíz cuadrada en el cuerpo R está definida solo para operandos no negativos.
- **Potenciación:** Requiere una condición de frontera cuando el exponente es nulo ($a^0 = 1$).

2.6. Ingeniería de Software Asistida por Modelos de Lenguaje

Un Modelo de Lenguaje de Gran Tamaño (LLM) es un sistema probabilístico entrenado para predecir secuencias de texto basándose en patrones preexistentes. Aunque sintetizan arquitecturas con rapidez, el código resultante debe auditarse bajo criterios de cohesión y bajo acoplamiento antes de su integración profesional [2].

3. Desarrollo

3.1. Ejercicio 1: Arquitectura del Editor Gráfico Generado por LLM

Para el primer ejercicio, se solicitó al LLM la generación de una gráfica modular en Java estructurada en tres componentes: la ventana principal, el lienzo interactivo y el registro de figuras geométricas.

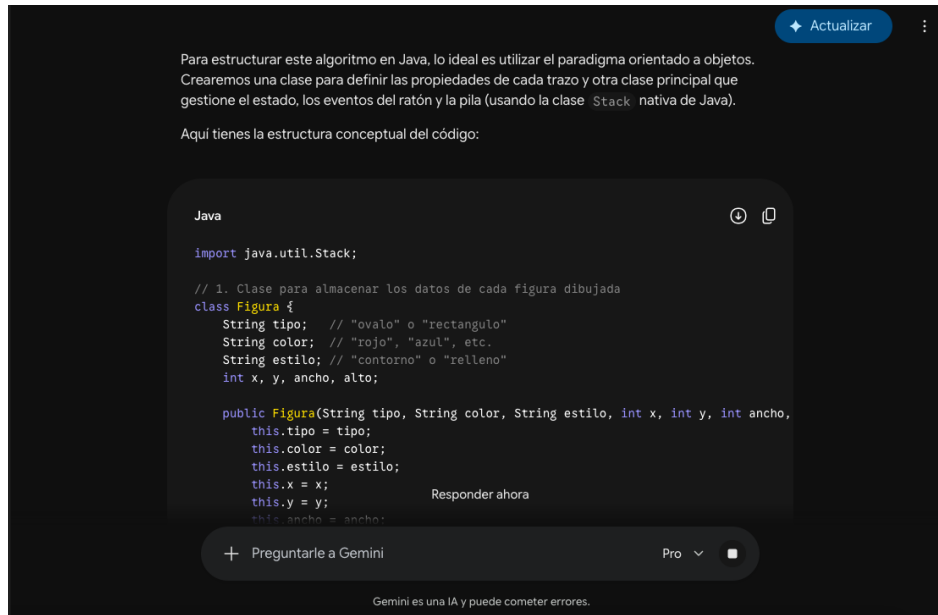


Figura 1: Respuesta de LLM.

1. **Modelado de Figuras:** La entidad geométrica se diseñó como un registro inmutable que almacena el tipo de figura, el punto de inicio, el punto de destino, el color de trazado y la bandera booleana de relleno. Esta inmutabilidad previene que una figura ya dibujada sea alterada accidentalmente por eventos posteriores del ratón.
2. **Captura de Eventos y Previsualización Dinámica:** En el panel del lienzo, se integró un adaptador de ratón que captura la coordenada de presión inicial. Durante el arrastre, el sistema genera una figura temporal de previsualización que se redibuja en tiempo real. Al liberar el botón izquierdo, se valida que las dimensiones sean mayores a un umbral mínimo para evitar artefactos visuales, incorporando la figura definitiva a la lista principal (Figura 2).
3. **Mecanismo de Deshacer (*Undo*):** Previo a cada mutación de la lista de figuras, el lienzo genera una copia profunda de la colección actual y la inserta en la cima de una pila estructurada sobre una cola de doble extremo. La operación de deshacer extrae la última instantánea válida y repinta el lienzo, garantizando la reversión no destructiva ilustrada en la Figura 3.
4. **Pipeline de Exportación Matricial:** Para persistir el dibujo, se genera un búfer de imagen en memoria con fondo blanco, se transfieren las primitivas mediante el contexto gráfico bidimensional y se serializa el mapa de bits hacia un archivo PNG en disco (Figura 4).

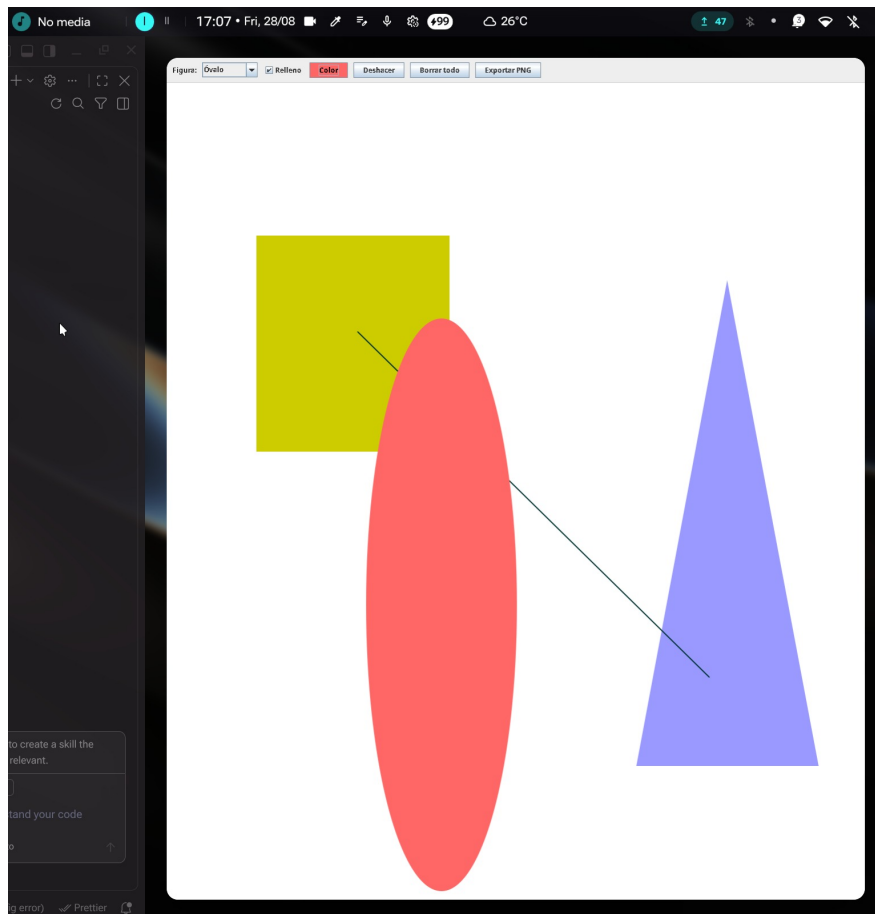


Figura 2: Interfaz de usuario del editor gráfico con figuras geométricas trazadas.

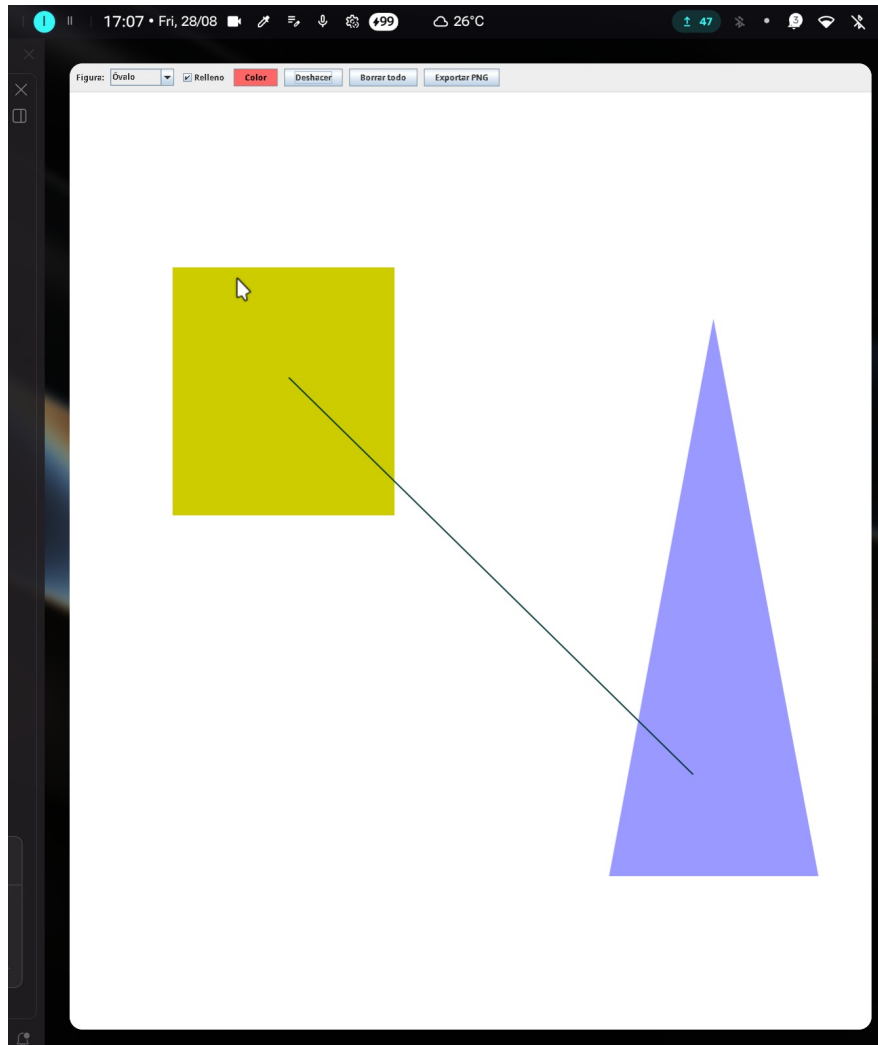


Figura 3: Evidencia visual del funcionamiento del mecanismo de reversión de estados.

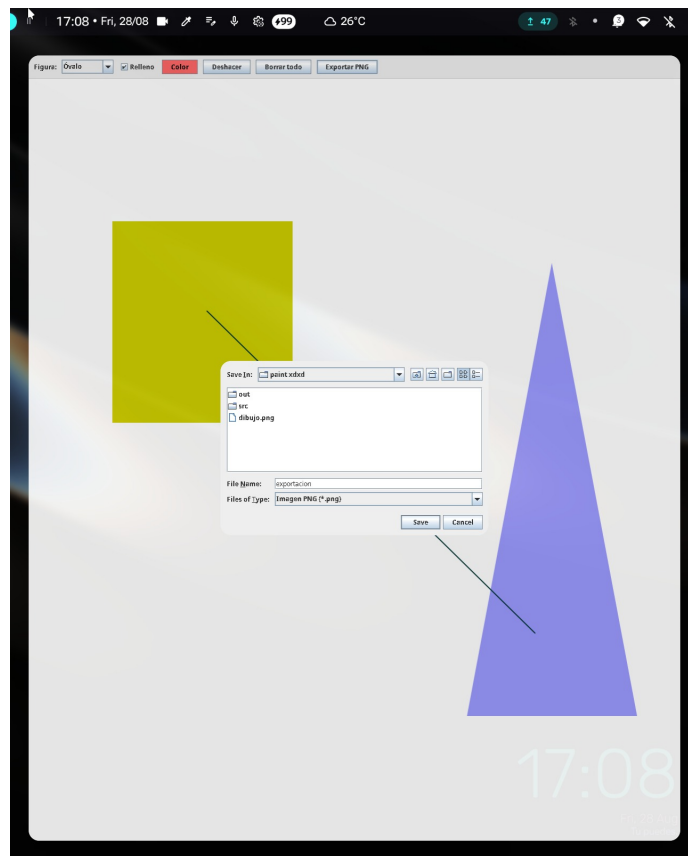


Figura 4: Imagen matricial exportada exitosamente desde el lienzo del editor.

3.2. Ejercicio 2: Formalización y Ejecución de la Lógica del Pizarrón

El segundo ejercicio consistió en traducir la lógica algorítmica y los diagramas de flujo colaborativos plasmados en el pizarrón del aula (Figura 5) hacia un programa interactivo monolítico y robusto en consola.

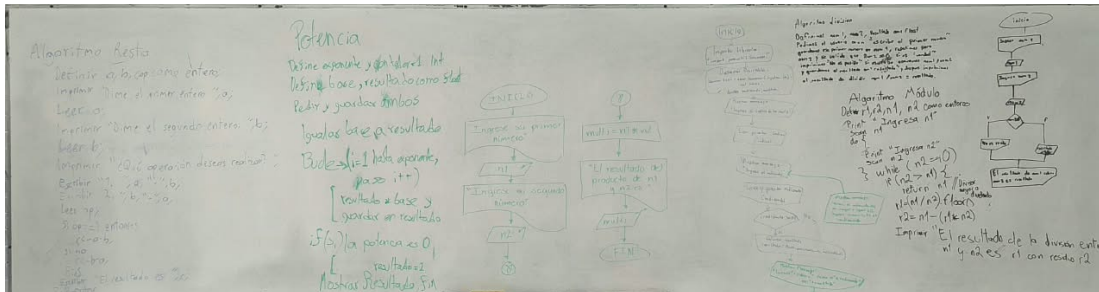


Figura 5: Fotografía del pizarrón con los algoritmos y diagramas colaborativos de clase.

3.2.1. Estructura Central de Despacho

El flujo principal del programa solicita al usuario la selección de la operación mediante un menú numérico. Esta entrada se evalúa a través de una estructura de selección múltiple (*switch-case*), la cual bifurca el flujo de control hacia la rutina específica correspondiente (Figura 6).

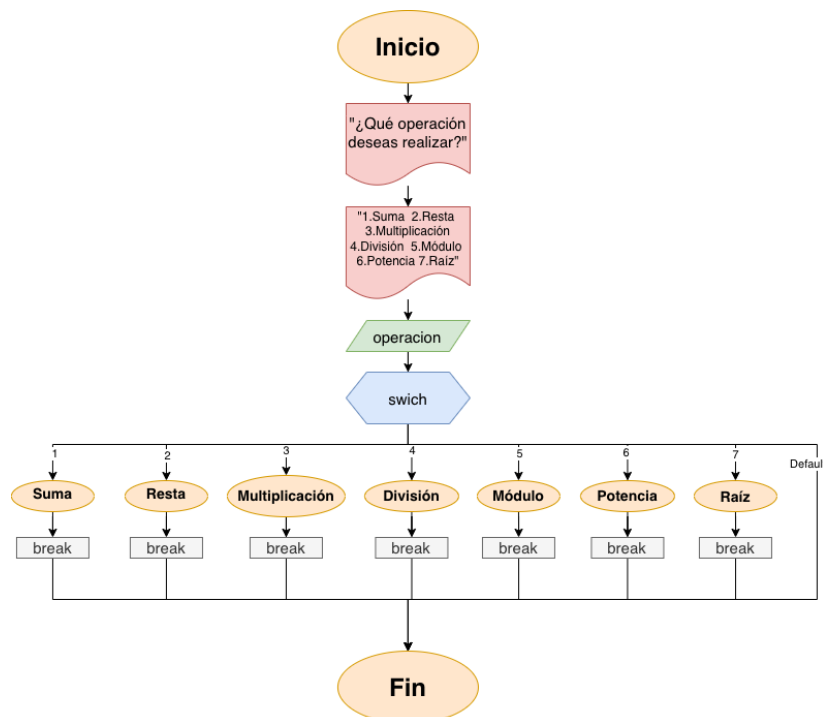


Figura 6: Diagrama de flujo consolidado del menú interactivo de la calculadora.

3.2.2. Operaciones Aritméticas

- **Adición y Sustracción:** Procesan operandos enteros mediante evaluación directa (Figuras 7 y 8).
- **Multiplicación:** Opera sobre números de punto flotante de doble precisión para soportar cálculos continuos (Figura 9).
- **Exponenciación:** En apego al diseño del pizarrón, se prescindió de bibliotecas matemáticas externas, implementando un ciclo iterativo que acumula el producto de la base tantas veces como indique el exponente entero, con evaluación defensiva para exponente cero (Figura 10).



Figura 7: Diagrama de flujo de la adición entera.



Figura 8: Diagrama de flujo de la sustracción entera.

3.2.3. Casos excepcionales de indeterminación

Las operaciones restantes demandaron un tratamiento riguroso de condiciones:

- **División:** Se interpone una compuerta de decisión antes del cálculo; si el denominador es cero, el programa emite una advertencia formal y suspende la evaluación para evitar indeterminaciones (Figura 11).
- **Módulo Manual:** Se implementó la deducción del residuo calculando el cociente entero $q = \lfloor a/b \rfloor$ y restando el producto $b \cdot q$ al dividendo original, validando previamente que el divisor no sea nulo (Figura 12).
- **Radicación Real:** Se verifica que el radicando sea no negativo previo a invocar la función de raíz cuadrada, notificando la inexistencia de solución real ante entradas negativas (Figura 13).

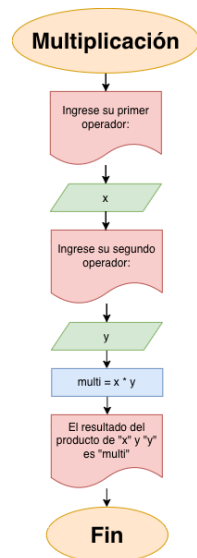


Figura 9: Diagrama de flujo del producto aritmético.

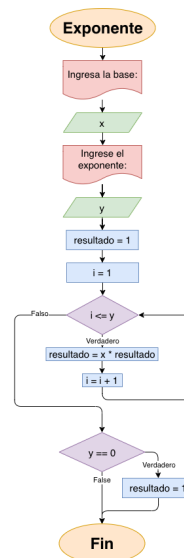


Figura 10: Diagrama de flujo de la potencia iterativa.

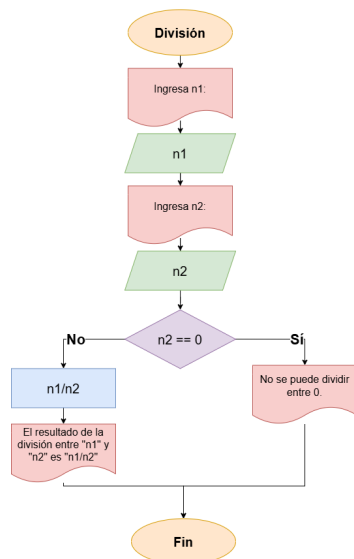


Figura 11: Diagrama de flujo de división controlada.

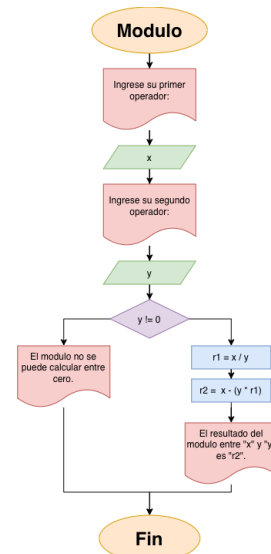


Figura 12: Diagrama de flujo de módulo manual.

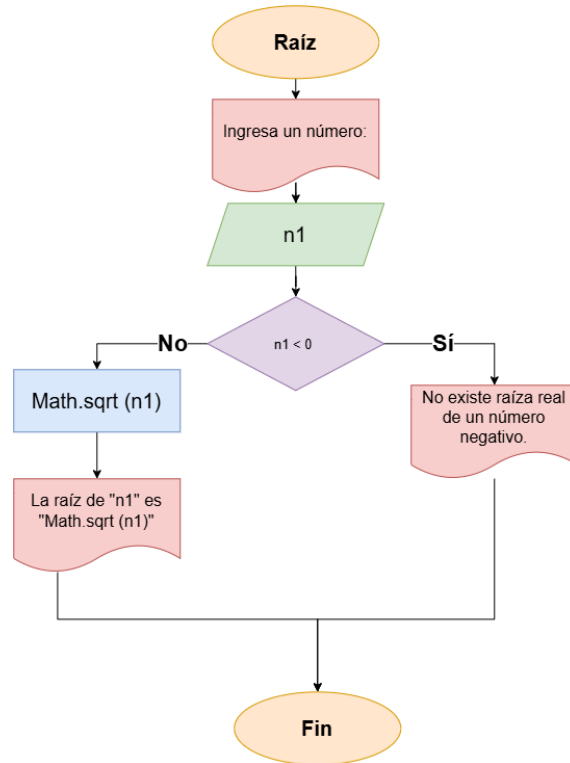


Figura 13: Diagrama de flujo de radicación con control de dominio.

4. Conclusión

La realización de esta práctica permitió contrastar dos paradigmas esenciales del desarrollo de software: la arquitectura reactiva basada en interfaces gráficas y el modelado de algoritmos en consola, el análisis técnico del editor gráfico demostró que los modelos de lenguaje son capaces de generar programas. No obstante, se evidenció que el implemento de su código, resulta en muchas ocasiones ofuscado a su fácil entendimiento.

En relación con los objetivos específicos planteados en la introducción, se dictamina el siguiente cumplimiento:

1. **Análisis de la arquitectura del editor gráfico generado por LLM:** *Cumplido.* Se examinó e ilustró formalmente el ciclo de vida de los componentes, la captura de coordenadas del ratón, la previsualización en tiempo real y la serialización a mapas de bits, sentando la línea base técnica para futuras comparativas.
2. **Implementación y verificación de la lógica del pizarrón:** *Cumplido.* La calculadora integró exitosamente todos los flujos diseñados por el grupo, bloqueando divisiones entre cero y raíces de negativos mediante bifurcaciones defensivas y resolviendo el residuo y la potencia mediante lógica manual explícita.

En conclusión, la práctica cumplió satisfactoriamente su propósito formativo al desacoplar el análisis conceptual de la implementación física, los artefactos desarrollados proporcionan una base empírica y metodológica sólida para abordar, en la siguiente etapa del curso, la comparación formal entre código

generado por inteligencia artificial y código elaborado íntegramente por desarrolladores humanos.

Referencias

- [1] Gamma, E., Helm, R., Johnson, R. y Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- [2] Deitel, P. J. y Deitel, H. M. (2017). *Java How to Program: Early Objects* (11.^a ed.). Pearson Education.
- [3] Schildt, H. (2018). *Java: The Complete Reference* (11.^a ed.). McGraw-Hill Education.
- [4] Sedgewick, R. y Wayne, K. (2011). *Algorithms* (4.^a ed.). Addison-Wesley.